3. Requirements Gathering

Initiative: Digital Egypt Pioneers Initiative (DEPI) – Round 5
Track: Microsoft Data Engineer
Company: Skills Dynamix
Group Code: ALX5_AIS5_S5
Project Name: Egyptian Ministry of Interior (MOI) Crime News Intelligence Pipeline
Supervisor / Lecturer: Eng. Ahmed Azab
Submission Date: October 16, 2026

Team Members

Member Name	**Project Role**	**Email Address**
Ahmed Ali Ahmed Ali	Team Leader	ahmed.a.ali.tech@gmail.com
Omar Tamer Mostafa Elhadidi	Data Engineer	omar.t.elhadidi@gmail.com
Mahmoud Ibrahim Abdul Qader	Data Engineer	mahmoud.ibrahim.eg1@gmail.com
Sondos Ahmed Mabrouk Habiba	Data Engineer	sondosahmed5425@gmail.com
Konoz Nashaat	Data Engineer	konoznashaat123@gmail.com
Shahd Nezar Atef Hagag	Data Engineer	shahdhagag.ss@gmail.com

Stakeholder Analysis

Stakeholder Group & Role	**Operational Requirements & Expectations**
Data Analysts & Researchers	• Clean, structured dataset exported in Parquet or CSV format. • Ability to slice and filter data by governorate, date range, and crime category. • Interactive visual charts and density heatmaps for reporting.

Stakeholder Group & Role	Operational Requirements & Expectations
Academic Supervisor (Eng. Ahmed Azab)	• Adherence to Medallion architecture (Bronze, Silver, Gold). • Fully automated daily orchestration via Apache Airflow. • Automated data quality invariant checks (`pytest`) and clean Star Schema design. • Clean Git repository structure and traceable commit history.
Engineering Team Members	• Modular codebase split into distinct folders (`src/`, `data/`, `sql/`, `tests/`, `dags/`). • Fully idempotent scripts that can be safely re-run without duplicating records. • Cloud-reproducible development environment (GitHub Codespaces).
General Public & Decision Makers	• Zero-setup web access via Streamlit browser app. • Intuitive user interface with fast filtering under 1 second. • Clear visual indicators with both Arabic and English labeling.

User Stories & Use Cases

User Stories

- **US-01 (Automated Ingestion):** *As a Data Engineer*, I want the pipeline to ingest new incident posts daily via an automated scheduler, so the data warehouse remains continuously updated without manual intervention.
  (Acceptance: Ingestion fetches only new posts, saves raw payload to Bronze JSON, and exits with code 0.)
- **US-02 (Geographic Incident Filtering):** *As a Safety Analyst*, I want to filter crime incidents by governorate and date range on an interactive map, so I can identify high-activity zones and seasonal patterns.
  (Acceptance: Selecting any of Egypt's 27 governorates dynamically updates metric cards and map coordinates within 1 second.)
- **US-03 (Data Quality Gate):** *As a Data Platform Engineer*, I want an automated test suite to validate warehouse integrity after every load, so that orphan foreign keys or duplicate records halt the pipeline before reaching the dashboard.
  (Acceptance: Test suite executes 5 pytest invariants; any failed assertion halts downstream tasks and logs errors.)
- **US-04 (Crime Category Trend Analysis):** *As a Researcher*, I want to view time-series breakdowns of specific crime types (e.g., narcotics, theft, cybercrime), so I can evaluate

the impact of law enforcement campaigns over time.
(Acceptance: Dashboard displays monthly distribution charts comparing year-over-year category volumes.)

Use Cases

Use Case 1: Automated Daily Batch Pipeline Execution

- **Primary Actor:** Apache Airflow Scheduler.
- **Precondition:** Airflow services are active; PostgreSQL database is reachable.
- **Execution Flow:**
    1. Airflow triggers `extract_crime_news.py`: pulls new Facebook posts into `data/bronze_crime_news.json`.
    2. Airflow triggers `transform_silver.py`: parses Arabic text, matches governorates, extracts categories, and saves columnar `data/silver_crime_news.parquet`.
    3. Airflow triggers `load_gold_warehouse.py`: performs batched upsert into PostgreSQL Star Schema using `execute_values` with `ON CONFLICT (news_id) DO NOTHING`.
    4. Airflow triggers `test_warehouse.py`: executes 5 pytest data quality invariants against the live database.
- **Postcondition:** Data warehouse updated with verified records; pipeline status logged as SUCCESS.

Use Case 2: Interactive Incident Exploration

- **Primary Actor:** End User / Data Analyst.
- **Precondition:** Streamlit application is deployed and running.
- **Execution Flow:**
    1. User opens the Streamlit web dashboard in a browser.
    2. User adjusts sidebar filters: selects governorate (e.g., Cairo, Alexandria, Giza), chooses date range, and picks crime category.
    3. Application queries clean dataset, calculates aggregation metrics, and re-renders KPI summary cards.
    4. Application renders interactive Egypt map with color-coded density markers at corresponding coordinates.
- **Postcondition:** Filtered visual insights and data tables rendered in under 1 second.

Functional Requirements

1. **FR-01 (Source Extraction):** The system must extract public crime news posts, capturing natural key `post_id`, publication timestamp `published_at`, raw Arabic narrative `raw_text`, and source URL.

2. **FR-02 (Arabic Text Cleaning & Extraction):** The transformation engine must clean diacritics, standardize punctuation, and extract: primary incident category (Narcotics, Weapons, Fraud, Theft, Cybercrime, Public Order), geographic governorate (mapped across all 27 Egyptian governorates), and incident metrics (arrest counts, weapons seized).
3. **FR-03 (Medallion Storage Architecture):** The system must implement three storage tiers: Bronze Layer (`data/bronze_crime_news.json`), Silver Layer (`data/silver_crime_news.parquet`), and Gold Layer (PostgreSQL Star Schema).
4. **FR-04 (Dimensional Star Schema Model):** The warehouse schema must consist of:
    - `DIM_DATE` (`date_key`, `full_date`, `year`, `month`, `month_name`, `quarter`, `day_of_week`, `is_weekend`)
    - `DIM_LOCATION` (`location_key`, `governorate_ar`, `governorate_en`, `latitude`, `longitude`, `region`)
    - `DIM_CRIME_TYPE` (`crime_type_key`, `category_name_ar`, `category_name_en`, `severity_level`)
    - `FACT_CRIME_NEWS` (`fact_key`, `news_id`, `date_key`, `location_key`, `crime_type_key`, `arrest_count`, `weapon_seized_flag`, `text_length`)
5. **FR-05 (Idempotent Database Loading):** The loading process must enforce natural key uniqueness on `news_id`. Re-running the pipeline on identical input must produce zero duplicate rows via `ON CONFLICT (news_id) DO NOTHING`.
6. **FR-06 (Airflow DAG Orchestration):** The system must define an Apache Airflow DAG (`dags/moi_crime_dag.py`) scheduled `@daily` with task dependencies (`extract >> transform >> load >> test`) and automatic retries.
7. **FR-07 (Automated Quality Gate):** The pipeline must include automated pytest verification checking: minimum volume threshold (≥ 4,700 records), zero foreign key orphans, natural key uniqueness, and valid date bounds.
8. **FR-08 (Interactive Dashboard Application):** The web application (`app.py`) must provide: overview KPI scorecards, interactive Egypt coordinate density heatmaps, and a filterable data table with CSV export.

Non-functional Requirements

1. **Performance & Throughput:** Batch transformation of 4,700+ records (Bronze to Silver) completes in under 3 minutes (> 1,500 records/sec); incremental daily runs complete in under 30 seconds; dashboard filter interactions re-render in under 1 second.
2. **Reliability, Fault Tolerance & Recovery:** Airflow tasks must include an automatic retry policy (2 retries with 5-minute backoff delay); database operations must run within atomic transactions; unclassified Arabic words must safely fall back to "Unclassified" without crashing.
3. **Data Integrity & Consistency:** 100% referential integrity with exactly 0 orphan records allowed between fact and dimension tables; columnar Parquet format must enforce strict data types.

4. **Usability & Accessibility:** The dashboard must be completely accessible through standard web browsers (Chrome, Edge, Firefox) without local installation; bilingual labels (Arabic/English) where appropriate.
5. **Maintainability & Modularity:** Clean directory structure (`src/`, `data/`, `sql/`, `tests/`, `dags/`) following PEP 8 conventions with type hints and docstrings.
6. **Security & Configuration Management:** All database credentials and connection parameters must be read from environment variables (`.env`); `.env` must be excluded from Git version control via `.gitignore`.